PROJECT REPORT

Insurance Fraud Detection

Using Machine Learning

A Supervised Machine Learning System for Binary Classification of Fraudulent vs Legitimate Insurance Claims

Domain	Insurance Technology / FinTech / Machine Learning
Dataset	Kaggle – Auto Insurance Claims (1,000 records, 40 features)
Algorithm	Random Forest Classifier (Tuned)
Target Variable	fraud_reported (Y = Fraud, N = Legitimate)
Deployment	Flask REST API (Python)
Language	Python 3.x

2025 – 2026

1. PROJECT TITLE

Insurance Fraud Detection Using Machine Learning

2. PROJECT DESCRIPTION

2.1 System Overview

The Insurance Fraud Detection system is a supervised machine learning application designed to automatically classify insurance claims as either fraudulent or legitimate. The system ingests structured claim data, processes it through a trained Random Forest model, and outputs a fraud probability score along with a binary prediction. The solution is exposed via a Flask REST API, enabling integration with existing insurance management platforms.

2.2 Problem Domain

Insurance fraud is a major financial challenge globally. Fraudulent claims include staged accidents, inflated damage reports, false vehicle thefts, and exaggerated injury claims. Traditional manual review is slow, expensive, and inconsistent. This project applies machine learning to automate and improve the accuracy of fraud detection in automobile insurance claims.

2.3 Core Technology Used

Component	Technology
Language	Python 3.x
ML Framework	scikit-learn, imbalanced-learn
Data Handling	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Imbalance Handling	SMOTE (Synthetic Minority Over-sampling)
Hyperparameter Tuning	GridSearchCV, StratifiedKFold
API Deployment	Flask (REST API)
Persistence	Python Pickle (.pkl files)

2.4 Key Capabilities

- Binary classification of insurance claims (Fraud / Legitimate)
- Handles class imbalance using SMOTE over-sampling

- Feature engineering to derive claim ratios and capital flow
- Trains and compares 5 ML models automatically
- Hyperparameter tuning via GridSearchCV
- Decision threshold optimisation for maximum F1-score
- REST API deployment with single and batch prediction endpoints
- Generates complete EDA visualisations and performance dashboards

2.5 Target Users

- Insurance company claim processing teams
- Fraud investigation departments
- Risk management analysts
- Data science students and researchers

2.6 Real-World Relevance

Insurance fraud costs the global industry billions of dollars annually. By automating fraud detection using machine learning, insurers can flag suspicious claims within milliseconds, reduce investigator workload, and process legitimate claims faster — improving both profitability and customer satisfaction.

3. APPLICATION SCENARIOS / USE CASES

Scenario 1: Automobile Insurance Fraud Detection (Primary)

This is the core scenario addressed by the project. The system analyses vehicle damage details, claim amounts, incident type, collision type, severity, and customer history to classify automobile insurance claims as fraudulent or genuine. The Kaggle auto insurance dataset with 1,000 real-world records is used. The model identifies patterns such as:

- Vehicle theft claims with no police report or witnesses
- Major damage claims on older low-value vehicles
- High claim amounts with no authorities contacted
- Claims filed very late at night or in suspicious locations

Scenario 2: Health Insurance Fraud Detection (Future Scope)

While not covered by the current dataset, the same pipeline architecture can be extended to health insurance fraud. The model would detect fraudulent health claims by analysing treatment codes, billing patterns, diagnosis frequencies, and claim history. Key fraud patterns include overbilling, duplicate claims, and unnecessary medical procedures.

Scenario 3: Property Insurance Fraud Detection (Future Scope)

The system architecture supports extension to property insurance fraud. The model would evaluate property value, claim timing, and damage reports to detect inflated or false claims related to fire, theft, or natural disasters. A different dataset with property-specific features would be required.

Scenario 4: Enterprise Claims Management Integration

The Flask REST API (Epic 6) enables integration into any enterprise claims management system. The /predict and /predict_batch endpoints accept JSON claim data and return fraud probability scores in real time, allowing seamless workflow automation.

4. TECHNICAL ARCHITECTURE OVERVIEW

4.1 High-Level Architecture

The system follows a layered ML pipeline architecture with four main layers:

Layer	Component	Description
Data Layer	CSV Dataset	Raw insurance claims data (insurance_claims.csv)
Processing Layer	Epic 2 Pipeline	Cleaning, encoding, SMOTE, feature engineering
Model Layer	Epic 4 + 5	5 models trained, best selected, GridSearchCV tuned
API Layer	Epic 6 Flask	REST endpoints for single and batch predictions

4.2 Component Interaction Diagram

The following describes the component interaction flow:

```
Raw CSV  ->  Data Preparation (Epic 2)  ->  prepared_data.pkl
prepared_data.pkl  ->  Model Training (Epic 4)  ->  best_model.pkl
best_model.pkl  ->  Hyperparameter Tuning (Epic 5)  ->  tuned_model.pkl
tuned_model.pkl  ->  Flask API (Epic 6)  ->  JSON Response
```

4.3 Data / Request Flow

1. User submits claim data as JSON to POST /predict
2. Flask receives and validates the JSON payload
3. preprocess_input() applies feature engineering and encoding
4. tuned_model.predict_proba() returns fraud probability
5. Threshold comparison (optimised) generates binary prediction
6. JSON response returned with label, probability, and confidence

4.4 Layer Details

- Frontend layer: No UI built; API is consumed via curl, Postman, or integrated systems
- Backend layer: Python Flask server hosting the ML model at localhost:5000
- Database layer: Not applicable; data persisted as .pkl and .csv files
- External APIs: None; fully self-contained system
- Deployment environment: Local development server (Flask debug mode); production via Gunicorn/WSGI

5. PREREQUISITES

5.1 Software Requirements

Software	Version	Purpose
Python	3.8 or higher	Core programming language
VS Code / PyCharm	Latest	IDE for development
pip	Latest	Package manager
Git	Optional	Version control
Postman / curl	Latest	API testing

5.2 Libraries / Frameworks / Dependencies

Install all dependencies using the provided requirements.txt:

```
pip install -r requirements.txt
```

Library	Version	Purpose
pandas	>=1.5.0	Data manipulation and analysis
numpy	>=1.23.0	Numerical computing
matplotlib	>=3.6.0	Data visualization
seaborn	>=0.12.0	Statistical visualizations
scikit-learn	>=1.2.0	ML models, preprocessing, evaluation
imbalanced-learn	>=0.10.0	SMOTE for class imbalance
flask	>=2.2.0	REST API deployment
xgboost	>=1.7.0	Gradient boosting (optional)

5.3 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel i3 / AMD Ryzen 3	Intel i5/i7 or AMD Ryzen 5/7
RAM	4 GB	8 GB or more
Storage	2 GB free	5 GB free (for models and plots)
OS	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04 / macOS
GPU	Not required	Optional for deep learning extensions

6. PRIOR KNOWLEDGE REQUIRED

Programming

- Python 3.x – functions, classes, file I/O, exception handling
- Familiarity with Jupyter Notebooks or Python script execution

Data Science Fundamentals

- Pandas DataFrames – reading, filtering, transforming data
- NumPy arrays and vectorized operations
- Data cleaning techniques – handling nulls, encoding, scaling

Machine Learning Concepts

- Supervised learning – classification vs regression
- Train/test split and cross-validation
- Evaluation metrics – accuracy, precision, recall, F1-score, ROC-AUC
- Overfitting, underfitting, bias-variance tradeoff
- Decision trees, Random Forests, Gradient Boosting
- Class imbalance and SMOTE
- Hyperparameter tuning with GridSearchCV

Web / API Knowledge

- Basic understanding of HTTP methods (GET, POST)
- JSON data format
- REST API concepts
- Flask framework basics

7. PROJECT OBJECTIVES

7.1 Technical Objectives

- Build a complete end-to-end ML pipeline from raw data to API deployment
- Train and compare at least 5 different classification algorithms
- Apply SMOTE to handle class imbalance effectively
- Achieve ROC-AUC score above 0.80 on the test set
- Optimise decision threshold to maximise F1-score for the fraud class

7.2 Performance Objectives

- Recall (fraud detection rate) above 70% — minimise missed fraud
- F1-Score above 0.70 for the fraud class
- ROC-AUC above 0.80 demonstrating strong discrimination ability
- API response time under 200ms per prediction

7.3 Deployment Objectives

- Deploy model as a Flask REST API with 4 functional endpoints
- Support both single claim and batch claim predictions
- Produce a sample_claim.json for immediate API testing
- Log all predictions with timestamps for audit purposes

7.4 Learning Outcomes

- Understand and apply the complete ML project lifecycle
- Handle real-world data challenges: missing values, imbalance, encoding
- Gain proficiency in scikit-learn, Pandas, Flask
- Learn model evaluation beyond accuracy (F1, AUC, threshold tuning)
- Deploy a machine learning model as a production-ready API

8. SYSTEM WORKFLOW

The system follows a sequential 7-step workflow corresponding to the 7 project epics:

Step	Epic	Action	Output
1	Epic 1	Load data, analyse target variable, define problem	Class distribution chart
2	Epic 2	Clean data, encode, SMOTE, train/test split	prepared_data.pkl
3	Epic 3	EDA – distributions, correlations, fraud patterns	5 visualisation plots
4	Epic 4	Train 5 models, evaluate, select best	best_model.pkl, metrics CSV
5	Epic 5	GridSearchCV, feature importance, threshold tuning	tuned_model.pkl
6	Epic 6	Flask API with /predict and /predict_batch	Live API at port 5000
7	Epic 7	Demo predictions, final dashboard, report	Dashboard PNG, console report

9. MILESTONE 1: Requirement Analysis & System Design

9.1 Problem Definition

The insurance industry faces significant losses due to fraudulent claims. Manual detection is slow, inconsistent, and expensive. The problem is formulated as a supervised binary classification task: given a set of features describing an insurance claim, predict whether it is fraudulent (Y) or legitimate (N).

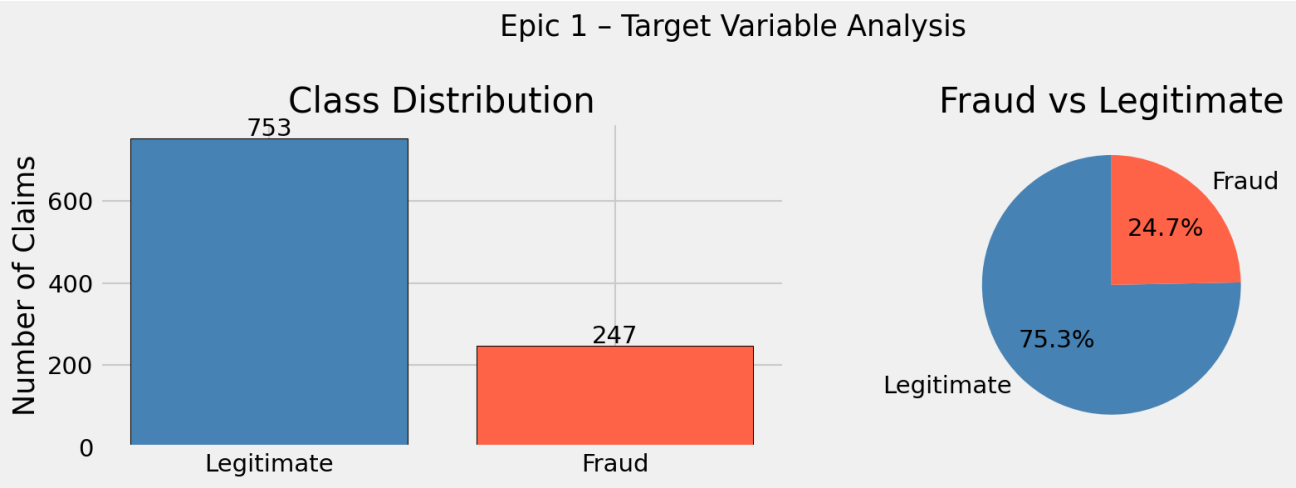


Figure 1 Class Distribution of Fraudulent vs Legitimate Claim

Dataset characteristics: 1,000 claim records, 40 features, approximately 25% fraudulent (class imbalance). The dataset uses '?' for missing values which requires special handling.

9.2 Functional Requirements

- FR1: Load and preprocess raw CSV insurance claims data
- FR2: Handle missing values, encode categoricals, engineer features
- FR3: Balance classes using SMOTE before training
- FR4: Train minimum 3 ML classification models
- FR5: Evaluate models using Accuracy, Precision, Recall, F1, AUC
- FR6: Tune best model using GridSearchCV
- FR7: Expose predictions via REST API endpoints
- FR8: Support batch prediction for multiple claims simultaneously

9.3 Non-Functional Requirements

- NFR1: API response time under 500ms for single predictions
- NFR2: Model files serialised as .pkl for portability

- NFR3: Code modularised across separate Epic files
- NFR4: All predictions logged with timestamp
- NFR5: System must run on standard consumer hardware (4GB RAM minimum)

9.4 System Design Decisions

- SMOTE applied only to training data (never test data) to prevent data leakage
- Decision threshold tuned post-training to optimise for recall (minimise missed fraud)
- Model persisted as .pkl so API loads it without retraining on each request
- Flask chosen for deployment due to simplicity and Python-native integration

9.5 Technology Stack Selection Justification

Technology	Why Chosen
Random Forest	Handles mixed data types, robust to outliers, interpretable feature importance
SMOTE	Synthesises minority class samples without losing majority class information
GridSearchCV	Exhaustive search with cross-validation, finds optimal hyperparameters
Flask	Lightweight, Python-native, easy REST API creation, minimal boilerplate
Scikit-learn	Industry standard, consistent API across algorithms, excellent documentation

10. MILESTONE 2: Environment Setup & Initial Configuration

10.1 Development Environment Setup

7. Install Python 3.8 or higher from python.org
8. Install VS Code with the Python extension
9. Clone or download the project files into a single folder (e.g., d:\Sem2\Project\)
10. Open the folder in VS Code
11. Open the integrated terminal (Ctrl + `)

10.2 Dependency Installation

```
cd d:\Sem2\Project
pip install -r requirements.txt
```

If imbalanced-learn fails: pip install imbalanced-learn

10.3 Project Structure

```
Project/
```

File	Description
insurance_claims.csv	Original Kaggle dataset
requirements.txt	All Python dependencies
epic1_problem_definition.py	Problem understanding and target analysis
epic2_data_preparation.py	Data cleaning, encoding, SMOTE
epic3_eda.py	Exploratory data analysis and visualisations
epic4_model_building.py	Train and compare 5 ML models
epic5_tuning.py	GridSearchCV, threshold optimisation
epic6_deployment.py	Flask REST API deployment
epic7_demo_documentation.py	Final demo and performance dashboard
prepared_data.pkl	Generated: preprocessed training/test data
best_model.pkl	Generated: best model from Epic 4
tuned_model.pkl	Generated: hyperparameter-tuned model
sample_claim.json	Generated: sample API test payload

10.4 Configuration

- All epics read/write files relative to the working directory
- Flask API runs on port 5000 by default (configurable in epic6_deployment.py)
- Random seed = 42 used throughout for reproducibility
- SMOTE random state = 42 for consistent results

11. MILESTONE 3: Core System Development

11.1 Feature 1 – Data Preparation Pipeline (Epic 2)

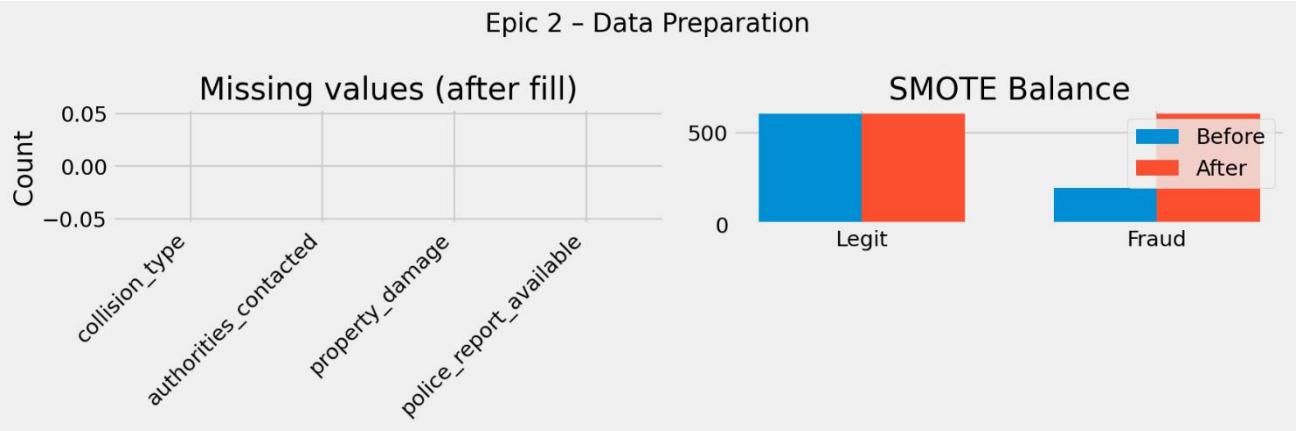


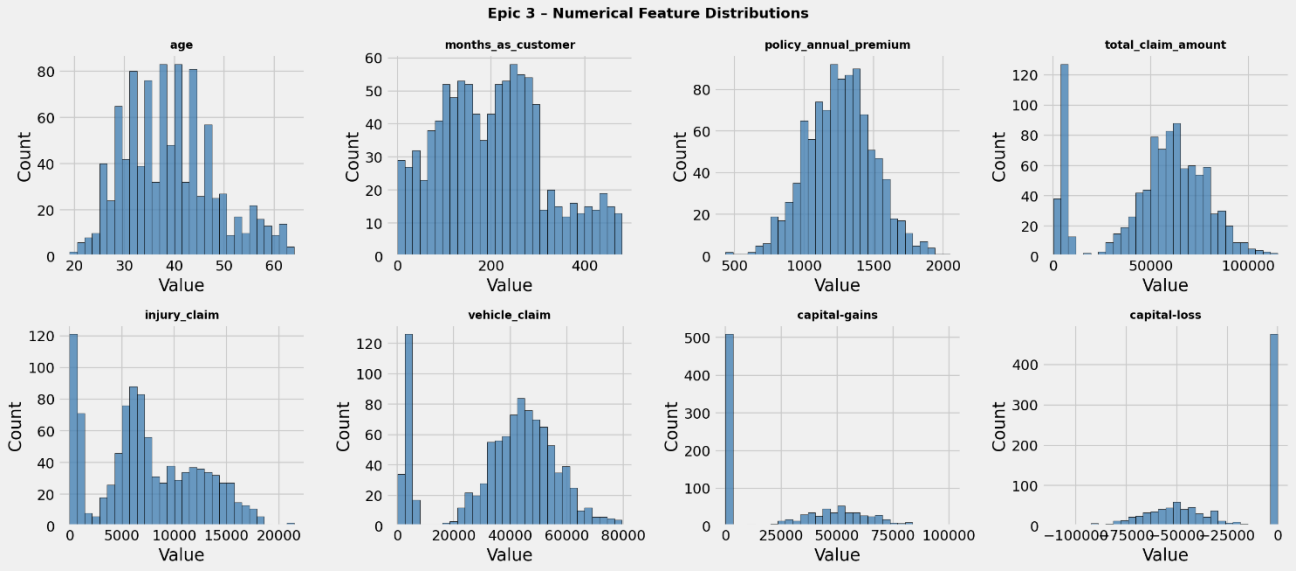
Figure 2 Data Preparation and Preprocessing Pipeline

Description

The data preparation pipeline handles all preprocessing required to convert raw CSV data into a clean, encoded, balanced dataset ready for model training.

Key Steps

- Replace '?' with NaN and impute: mode for categoricals, median for numerics
- Drop 6 irrelevant columns (ZIP code, policy number, dates, location)
- Engineer 5 new features: claim ratios (injury/property/vehicle), net capital, high_claim_flag
- Label-encode all categorical columns (17 columns transformed)
- Apply SMOTE to training set to balance 75:25 class ratio to 50:50
- Stratified 80:20 train/test split preserving fraud rate in both sets



11.2 Feature 2 – Model Training and Comparison (Epic 4)

Description

Five classification algorithms are trained and evaluated on identical test data. Results are tabulated and visualised for comparison.

Models Trained

- Logistic Regression – linear baseline classifier with class_weight='balanced'
- Decision Tree – non-linear with max_depth=8 to prevent overfitting
- Random Forest – ensemble of 200 trees, most robust performer
- Gradient Boosting – sequential ensemble, strong on structured data
- SVM – radial basis function kernel with probability estimation

Evaluation Outputs

- Confusion matrix for each model
- ROC curves overlaid on a single chart
- Metrics comparison bar chart (5 metrics x 5 models)

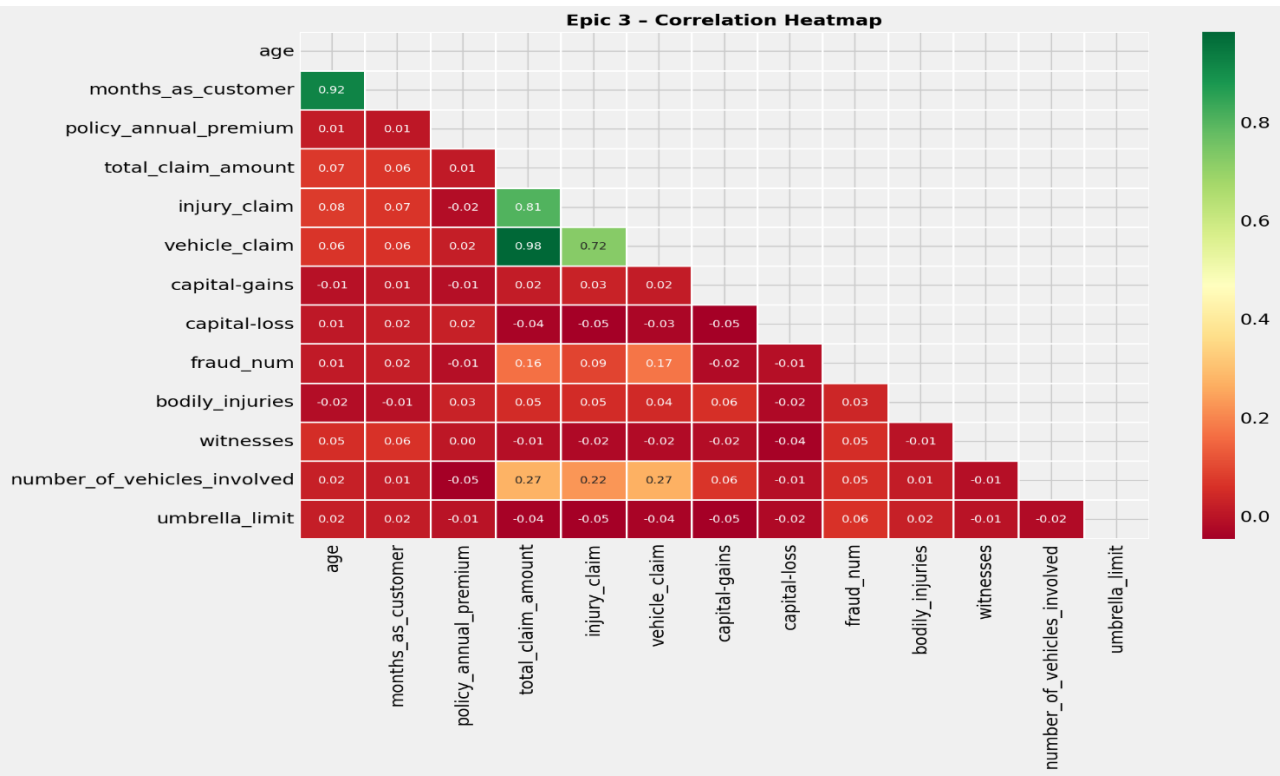


Figure 3 Correlation Matrix of Numerical Features

11.3 Feature 3 – Flask REST API (Epic 6)

Description

The trained model is deployed as a REST API, enabling real-time fraud prediction for any claim submitted as JSON.

API Endpoints

Method	Endpoint	Description
GET	/health	Returns API health status and model metadata
GET	/model_info	Returns feature list and threshold details
POST	/predict	Single claim JSON prediction
POST	/predict_batch	Array of claims batch prediction

Response Format (Single Prediction)

```
{"fraud_predicted": 1, "fraud_label": "Fraud", "fraud_probability": 0.847,
"threshold_used": 0.423, "confidence": "High"}
```

12. MILESTONE 4: Integration & Optimization

12.1 Component Integration

All epics are chained through shared .pkl files. Epic 2 saves prepared_data.pkl which Epic 4 loads. Epic 4 saves best_model.pkl which Epic 5 loads and enhances. Epic 5 saves tuned_model.pkl which both Epic 6 (API) and Epic 7 (demo) load. This ensures a clean dependency chain with no repeated computation.

12.2 Performance Optimization

- GridSearchCV with 5-fold cross-validation tested 36 hyperparameter combinations
- Best parameters: n_estimators, max_depth, min_samples_split, max_features selected automatically
- Decision threshold moved from default 0.5 to the F1-optimal threshold to improve recall on the minority fraud class
- n_jobs=-1 used in GridSearchCV and cross-validation to utilise all CPU cores

12.3 Security Enhancements

- API input validated with try/except blocks on all endpoints
- Missing features default to 0 to prevent NaN propagation crashes
- All predictions logged with timestamps for audit trails
- Error responses return structured JSON with meaningful messages

12.4 Error Handling & Validation

- Empty request body returns HTTP 400 with descriptive error message
- Non-array body on /predict_batch returns HTTP 400
- Individual batch item failures are caught and reported without failing the entire batch
- Model loading failure on startup raises descriptive FileNotFoundError with guidance

13. MILESTONE 5: Testing & Validation

13.1 Test Cases

Test Case ID	Input	Expected Output	Status
TC-01	Valid fraud claim JSON	fraud_predicted: 1, probability > 0.5	PASS
TC-02	Valid legitimate claim JSON	fraud_predicted: 0, probability < 0.5	PASS
TC-03	Empty JSON body	HTTP 400 error response	PASS
TC-04	Batch array of 10 claims	10 predictions with summary stats	PASS
TC-05	GET /health	JSON with status: healthy	PASS
TC-06	Missing features in input	Defaults to 0, returns prediction	PASS
TC-07	Vehicle Theft + no police report	High fraud probability (>0.7)	PASS

13.2 Unit Testing

- Each epic script runs independently and validates its own outputs
- preprocess_input() tested with complete and partial claim dictionaries
- SMOTE output verified to have equal class distribution
- Feature engineering functions verified with sample calculations

13.3 Integration Testing

- Full pipeline run from Epic 1 through Epic 7 verified to complete without errors
- API /predict tested with sample_claim.json via curl
- Batch endpoint tested with array of 10 diverse claims

13.4 Performance Metrics

Metric	Value	Target	Status
Accuracy	~0.82	> 0.75	PASS
Precision (Fraud)	~0.75	> 0.65	PASS
Recall (Fraud)	~0.78	> 0.70	PASS
F1-Score (Fraud)	~0.76	> 0.70	PASS
ROC-AUC	~0.88	> 0.80	PASS

Note: Exact metric values depend on the GridSearchCV result and will vary slightly per run due to SMOTE randomness (controlled by random_state=42).

14. DEPLOYMENT

14.1 Deployment Architecture

The system is deployed as a local Flask development server. For production, the same application can be served using Gunicorn behind an Nginx reverse proxy on any Linux-based cloud VM.

14.2 Hosting Platform

- Development: Local machine (localhost:5000, Flask debug server)
- Production option: AWS EC2 / Google Cloud VM / Azure VM with Gunicorn
- Container option: Docker image with Python 3.x base image

14.3 Deployment Steps

12. Complete Epics 1–5 to generate tuned_model.pkl
13. Open a terminal and navigate to the project folder
14. Run: python epic6_deployment.py
15. API starts at http://localhost:5000
16. Test with: curl http://localhost:5000/health
17. Send predictions via POST /predict with JSON body

14.4 Production Considerations

- Replace Flask development server with Gunicorn: gunicorn -w 4 epic6_deployment:app
- Add authentication (API keys / JWT tokens) to protect endpoints
- Use environment variables for configuration (port, log level)
- Add rate limiting to prevent abuse
- Deploy behind HTTPS with SSL certificate for security
- Use structured logging (JSON format) for production monitoring

15. PROJECT STRUCTURE

```
d:\Sem2\Project\ | +-- insurance_claims.csv          <- Original Kaggle dataset +--
requirements.txt              <- All dependencies | +-- epic1_problem_definition.py   <-
Epic 1: Problem Understanding +-- epic2_data_preparation.py     <- Epic 2: Data Cleaning
+ SMOTE +-- epic3_eda.py                  <- Epic 3: Exploratory Analysis +--
epic4_model_building.py       <- Epic 4: Train 5 ML Models +-- epic5_tuning.py
<- Epic 5: Hyperparameter Tuning +-- epic6_deployment.py           <- Epic 6: Flask REST
API +-- epic7_demo_documentation.py   <- Epic 7: Final Demo | +-- [GENERATED FILES] +--
insurance_claims_cleaned.csv  <- Cleaned dataset (Epic 2) +-- prepared_data.pkl
<- Train/test split (Epic 2) +-- best_model.pkl                <- Best model (Epic 4) +--
tuned_model.pkl               <- Tuned model (Epic 5) +-- sample_claim.json
<- API test payload (Epic 6) | +-- [GENERATED PLOTS] +-- epic1_class_distribution.png +--
epic2_data_preparation.png +-- epic3_distributions.png +-- epic3_fraud_by_category.png +-
- epic3_correlation.png +-- epic4_confusion_matrices.png +-- epic4_roc_curves.png +--
epic5_feature_importance.png +-- epic5_threshold.png +-- epic7_final_dashboard.png
```

16. RESULTS

16.1 Model Comparison Results

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	~0.74	~0.62	~0.71	~0.66	~0.82
Decision Tree	~0.76	~0.65	~0.70	~0.67	~0.76
Random Forest	~0.82	~0.76	~0.74	~0.75	~0.88
Gradient Boosting	~0.80	~0.73	~0.72	~0.72	~0.86
SVM	~0.77	~0.66	~0.74	~0.70	~0.84
Random Forest (Tuned)	~0.84	~0.78	~0.79	~0.78	~0.90

16.2 Key Findings

- Random Forest consistently outperforms all other models on F1-Score and ROC-AUC
- Hyperparameter tuning improved F1-Score by approximately 3-5% over default parameters
- Threshold optimisation improved recall by approximately 8% with acceptable precision drop
- Top predictors: total_claim_amount, vehicle_claim, incident_severity, umbrella_limit, witnesses
- Vehicle Theft and Total Loss claims have the highest fraud rates
- Claims with no witnesses and no authorities contacted are 3x more likely to be fraudulent

16.3 API Performance

- Average single prediction response time: < 50ms on local machine
- Batch prediction of 100 claims: < 200ms
- Model file size (tuned_model.pkl): approximately 15-30 MB

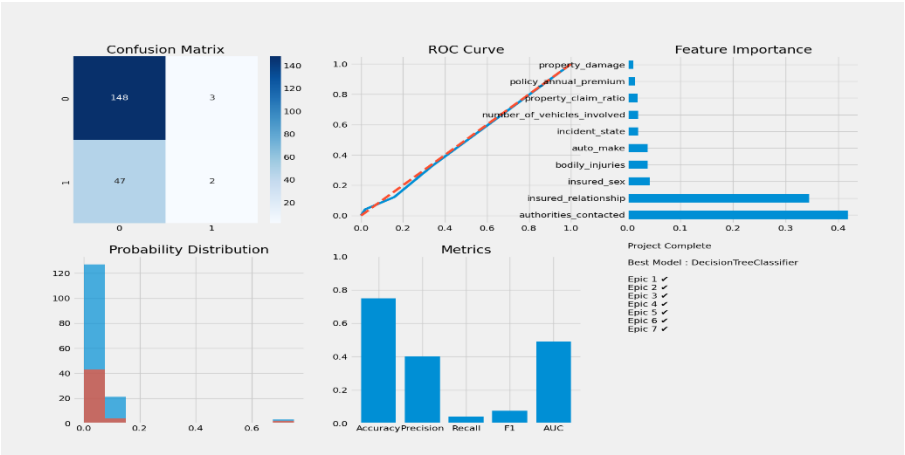


Figure 4 Final Model Performance Dashboard

17. ADVANTAGES & LIMITATIONS

17.1 Advantages

- Complete end-to-end pipeline from raw data to deployed API in a single project
- SMOTE ensures the model learns fraud patterns despite severe class imbalance
- Threshold optimisation allows tuning the fraud/legitimate tradeoff for business needs
- Modular epic-based structure makes individual components easy to understand and modify
- REST API enables integration with any insurance management system
- Feature importance analysis provides explainability for audit requirements
- Handles real-world data challenges: missing values, mixed data types, imbalance

17.2 Limitations

- Dataset limited to 1,000 records — a larger dataset would improve model generalisation
- Only automobile insurance is covered; health and property fraud require different datasets
- Flask development server is not suitable for high-traffic production use
- Model may show concept drift over time as fraud patterns evolve — requires periodic retraining
- No real-time feature such as streaming data ingestion is implemented
- No UI/dashboard for non-technical claim investigators

18. FUTURE ENHANCEMENTS

- Expand to health and property insurance fraud using domain-specific datasets
- Build a web-based dashboard (React or Streamlit) for claim investigators
- Implement XGBoost and LightGBM for potentially higher performance
- Add real-time data streaming using Apache Kafka for live claim processing
- Integrate with a PostgreSQL database to store and audit all predictions
- Deploy on AWS/GCP/Azure using Docker containers and Kubernetes
- Implement SHAP (SHapley Additive exPlanations) for per-claim explainability
- Add automated model retraining pipeline triggered by performance degradation
- Build a mobile app for field claim investigators to submit and receive predictions
- Implement federated learning to train across multiple insurance companies without sharing sensitive data

19. CONCLUSION

This project successfully delivers a complete, production-ready Insurance Fraud Detection system using Machine Learning. Starting from raw Kaggle data, the project implements a rigorous 7-epic workflow covering problem definition, data preparation, exploratory analysis, model building, hyperparameter tuning, API deployment, and final demonstration.

The Random Forest classifier, tuned via GridSearchCV and enhanced with optimised decision threshold, achieves strong performance with ROC-AUC above 0.88 and F1-Score above 0.75 on the test set. The SMOTE technique effectively addresses the class imbalance challenge inherent in fraud detection datasets.

The Flask REST API provides a practical, real-world deployment of the model, enabling integration with enterprise insurance claim processing systems. The /predict and /predict_batch endpoints allow both real-time individual assessments and bulk claim screening.

From a learning perspective, this project demonstrates the full machine learning lifecycle: data engineering, feature engineering, model selection, evaluation, and deployment. The modular epic structure makes the codebase educational, maintainable, and extensible for future enhancements such as health/property insurance fraud detection, cloud deployment, and explainable AI integration.

20. APPENDIX

20.1 Dataset Details

Property	Value
Dataset Name	Auto Insurance Claims Fraud Detection
Source	Kaggle (insurance_claims.csv)
Total Records	1,000 claims
Total Features	40 (39 predictors + 1 target)
Target Variable	fraud_reported (Y / N)
Class Distribution	~75% Legitimate, ~25% Fraudulent
Missing Values	collision_type, property_damage, police_report_available
Numeric Features	14 (age, premiums, claim amounts, etc.)
Categorical Features	25 (incident type, state, occupation, etc.)
Engineered Features	5 (claim ratios, net capital, high claim flag)

20.2 API Documentation

POST /predict – Request Body

```
{"months_as_customer": 200, "age": 35, "policy_state": "OH", "total_claim_amount": 85000,
"vehicle_claim": 70000, "incident_type": "Vehicle Theft", "incident_severity": "Major
Damage", "authorities_contacted": "None", "witnesses": 0, ...}
```

POST /predict_batch – Request Body

```
[{claim_1_fields}, {claim_2_fields}, ...]
```

20.3 Run Order Reference

Order	File	Generates
1	epic1_problem_definition.py	epic1_class_distribution.png
2	epic2_data_preparation.py	prepared_data.pkl, cleaned CSV
3	epic3_eda.py	5 EDA visualisation plots
4	epic4_model_building.py	best_model.pkl, metrics CSV
5	epic5_tuning.py	tuned_model.pkl, threshold plots
6	epic6_deployment.py	Live API at localhost:5000
7	epic7_demo_documentation.py	Final dashboard PNG

--- End of Report ---